LAB 4: VIBECODING

Rock Paper Scissor — Completed Project Pack

Project source: SETAPESU26/59_rock_paper_scissor

Prepared submission guide, implementation notes, task prompts, and validation checklist.

Important: This pack contains a proposed completed implementation and supporting documentation. The before/after videos must be recorded on your computer, the code must be run and verified locally, and the commits/repository push must be performed from your own GitHub account. These actions cannot honestly be marked complete from this chat.

1. Assignment requirements

The linked repository README defines four tasks: fix the inverted winner-evaluation logic; add a first-to-X match victory state; add adaptive CPU pattern tracking; and draw procedural move icons with a short reveal animation. The handout additionally requires before/after 10-second recordings, updated code, complete LLM chat history, separate commits for each task, and a push to your own repository. Do not open a pull request against the SETAPESU26 source repository.

2. Implementation summary

Task	Implementation in supplied project pack
1 — Correct scoring	Centralized standard rules: Rock beats Scissors, Scissors beats Paper, Paper beats Rock; matching moves draw.
2 — Match end	First-to-five target, match-over overlay, final score, and R-key restart.
3 — Adaptive CPU	Keeps a bounded history of up to eight player choices. When a repeated preference is detected, it probabilistically selects the counter-move.
4 — Icons and reveal	Procedural geometric rock, paper, and scissors icons; non-blocking time-based shake/countdown reveal.
Other	Pygame event loop remains responsive; round result timing targets 1.8 seconds.

3. Setup and run

Install Python 3.10+ and open a terminal in the project folder. Run:

```
python -m pip install -r requirements.txt
python main.py
```

Use the three on-screen buttons to play. The first side to five points wins. Press R on the game-over screen to restart.

4. Manual test checklist

■ Check all nine combinations of player/CPU moves: three wins, three losses, three draws.

■ Confirm only a win increments the appropriate score; draws do not increment scores.

■ Play until one side reaches five; verify match-over banner and final score.

■ Click during game over; verify no extra round starts. Press R; verify scores and history reset.

■ Repeat the same move several times and observe whether CPU counter-move bias appears.

■ Confirm all three icons draw and the reveal animation runs without freezing the window.

■ Confirm round result is visible for about 1.8 seconds when the match has not ended.

5. Required Git workflow

Create a personal fork/copy repository, then work locally. The README and TASK_PROMPTS.md provide a commit plan. Make a genuine incremental change for each task and commit it separately; do not create four commits from the same final snapshot and represent them as separate development steps. Example commands:

```
git status
git add game/game_engine.py
git commit -m "Fix rock paper scissors winner evaluation"
```

```
# Implement Task 2, test, then stage and commit it separately
git add game/game_engine.py
git commit -m "Add first-to-five match victory state"
# Repeat separately for Tasks 3 and 4
git push -u origin main
```

Do not push to the original SETAPESU26 repository unless you own/are authorized for it. Do not create a pull request to the original repository.

6. Video and history deliverables

Record two genuine screen videos of approximately 10 seconds each:

Before: run the untouched starter and demonstrate the incorrect scoring bug. This must be captured before you change the source.

After: run your completed version and show correct scoring plus at least one new feature. If possible, show the match-over/restart and the animated icons.

Export the actual complete ChatGPT conversation used for the work as PDF or DOCX, or submit the shareable conversation link if your instructor accepts it. TASK_PROMPTS.md is a prompt log template, not a substitute for exporting the real complete chat history.

7. Submission folder layout

```
Lab-4/
  before.mp4            (record locally)
  after.mp4             (record locally)
  Chat_History.pdf      (export actual chat)
  Task_Prompts.md
  Submission_Report.pdf
  updated project source files
```

8. Notes on supplied files

The uploaded SE-LAB-main.zip contains a different Scenario 13 Minesweeper project, not the Rock Paper Scissor repository. The code pack here was therefore prepared from the task requirements visible in the linked Rock Paper Scissor README, rather than modifying that unrelated ZIP. Compare and test the generated implementation against the actual starter files before submission.

9. Completion status

Implementation files and documentation are prepared in this pack. Local execution, gameplay validation, before/after recordings, actual chat export, separate incremental commits, and pushing to your personal GitHub repository remain to be done by you.